Лабораторная работа №3.
Применение каскадных таблиц стилей при создании web-сайта

Цель работы: изучить основы синтаксиса каскадных таблиц стилей, применить полученные знания для написания сайтов.

Теоретический материал

Использование каскадных таблиц стилей уже само по себе повышает удобство разработки сайта. Активное применение стилей и управление с помощью них видом элементов веб-страниц — современная парадигма создания сайтов. Именно по этой причине следует уделить достаточно внимания написанию стилевого кода. Учитывая некоторые особенности которые описаны ниже, можно создать действительно эффективный и удобный документ, содержащий описание стиля и ускоряющий загрузку веб-страниц.

CSS – это язык для определения того, как документы представляются пользователям – как они оформляются, размещаются и т.д.

Документ обычно представляет собой текстовый файл, структурированный с использованием языка разметки: HTML – самый распространенный язык разметки, но Вы также можете встретить другие языки разметки, такие как SVG или XML.

Представление документа пользователю означает преобразование его в форму, используемую Вашей аудиторией**.** Browsers, такие как Firefox, Chrome или Edge, предназначены для визуального представления документов, например, на экране компьютера, проектора или принтера.

Браузер иногда называют user agent, что в основном означает компьютерную программу, которая представляет человека внутри компьютерной системы. Браузеры – это основной тип пользовательских агентов, которые мы ассоциируем с CSS, но он не единственный. Доступны и другие пользовательские агенты, например, те, которые преобразуют документы HTML и CSS в файлы PDF для печати.

CSS может использоваться для очень простой стилизации текста документа, например, для изменения цвета и размера заголовков и ссылок. Он может быть использован для создания макета, например, превращения одного столбца текста в макет с основной областью контента и боковой панелью для соответствующей информации. Он также может использоваться для создания эффектов, таких как анимация.

CSS – это язык на основе правил: Вы задаёте правила, определяющие группы стилей, которые должны применяться к определённым элементам или группам элементов на Вашей веб-странице.

Все технологии веб-стандартов (HTML, CSS, JavaScript и т. д.) определены в гигантских документах, называемых спецификациями, которые публикуются организациями по стандартизации (такими как W3C, WHATWG, ECMA или Khronos) и определяют, как эти технологии должны себя вести.

CSS разработан группой в W3C, которая называется CSS Working Group. Эта группа состоит из представителей производителей браузеров и других компаний, которые заинтересованы в CSS. Есть также другие люди, известные как приглашенные эксперты, которые выступают как независимые голоса; они не связаны с членами организации.

Новые функции CSS разрабатываются или определяются рабочей группой CSS, иногда потому, что конкретный производитель браузера заинтересован в том, чтобы иметь какие-то возможности, иногда потому, что веб-дизайнеры и разработчики запрашивают функцию, а иногда потому, что сама рабочая группа определила требование. CSS постоянно развивается, появляются новые функции. Тем не менее, ключевым моментом в CSS является то, что никогда изменения не должны приводить к тому, чтобы перестали работать созданные до их появления сайты. Веб-сайт, созданный в 2000 году, с использованием ограниченного CSS, доступного в то время, должен всё ещё использоваться в браузере и сегодня!

CSS-спецификации огромны – они предназначены для инженеров, для реализации поддержки функций в пользовательских агентах, а не для веб-разработчиков, чтобы читать и понимать CSS. Многие опытные разработчики предпочитают обращаться к специальной документации. Однако стоит знать, что они существуют для того, чтобы понимать взаимосвязь между используемым CSS, поддержкой браузера и спецификациями.

Использование CSS имеет смысл только, если один или несколько браузеров его реализовали. Это означает, что код был написан для превращения инструкции в нашем CSS-файле во что-то, что может быть выведено на экран. Зачастую некоторая часть CSS-инструкций работает в одних браузерах, но не работает в других. По этой причине при разработке полезно проверять состояние реализации.

Размещение стилей в отдельном файле позволяет ускорить загрузку веб-страниц за счет уменьшения кода, а также кэширования файла с описанием стиля. Также файл со стилями позволяет, изменяя параметры в одном единственном месте, управлять видом всех страниц, где указана на него ссылка.

Селекторы Class удобно использовать, когда нужно применить стиль к разным тегам веб-страницы: ячейкам таблицы, ссылкам, параграфам и др. Для изменения отдельных слов или даже букв, а также блоков, содержащих в себе разные элементы употребляются теги SPAN и DIV.
Тег SPAN предназначен для изменения стиля выделенной части текста, такой как отдельный символ или их набор, а DIV позволяет применять стиль сразу к целому блоку. Перед этим тегом, в отличие от SPAN, автоматически добавляется перенос строки и пустое пространство, аналогично действию параграфа.

Несколько подряд идущих тегов, задающих форматирование, лучше заменить одним тегом SPAN с заданным стилем.

Пока происходит работа над сайтом, файл со стилями приходится изменять по многу раз. Для повышения удобства и понимания цели

различных селекторов, используйте комментарий, который помещается между символами /* и */. Комментарий повышает читабельность кода и позволяет легко разобраться в нем даже по прошествии некоторого времени. Другим способом создания удобного стиля является использование различных отступов и пробелов. Лучше для этой цели подходит знак табуляции, он добавляет сразу несколько пробелов, и при этом занимает минимум памяти. Сравните два способа описания параметров (пример 1).

Пример 1. Написание файла со стилем

```
/* Данный стиль краткий, но неудобный для внесения изменений */

a {font-family: Arial, Helvetica, sans-serif; font-size: 80%; color: #003366;}
a:visited {color: #660066;}
a:hover {color: #800000;}
.title {font-size: 120%; font-family: Verdana, Arial, Helvetica, sans-serif; color:
#FFFFEE; text-align: center;}


/* Тот же самый стиль, но представлен более «красиво» и понятно */

a {
font-family: Arial, Helvetica, sans-serif;
font-size: 80%;
color: #003366;
}

a:visited {
color: #660066;
}

a:hover {
color: #800000;
}

.title {
font-size: 120%;
font-family: Verdana, Arial, Helvetica, sans-serif;
color: #FFFFEE;
text-align: center; }
```

Несмотря на то, что объем нижнего примера больше, чем у верхнего, управлять параметрами стилей удобней и наглядней.

«Идеальный» код HTML должен содержать только теги и текст, а более ничего лишнего. Для достижения такого результата или хотя бы приближения к нему, описание стилей тегов выносится в отдельный файл, а сам тег пишется без всяких параметров (пример 2). В этом случае происходит сокращение кода и ускорение загрузки веб-страниц.

Пример 2. Подключение таблицы связанных стилей

```
<html>
<head>
<link rel="stylesheet" type="text/css" href="mysite.css">
или
<link rel="stylesheet" type="text/css" href="http://www.mysite.ru/main.css">
</head>
<body>
<h1>Hello, world!</h1>
</body>
</html>
```

Пример содержимого файла (например, mysite.css):

```
body {
font-family: 'Times New Roman', serif;
font-size: 12pt;
color: darkgreen;
}
h1 {
font-family: Arial, sans-serif;
font-size: 16pt;
color: green;
font-weight: bold;
}
h2 {
font-family: Arial, sans-serif;
font-size: 14pt;
color: greenyellow;
font-weight: bold;
font-style: italic;
}
```

Обратите внимание: теги <style> и </style> внутри файла таблицы стилей не используются – расширение .css явно указывает браузеру на то, что файл является таблицей стилей. Один такой файл может быть связан сразу с несколькими страницами (или импортирован сразу в несколько страниц). В html-файле для связывания нужно прописать в любом месте между тегами <head> и </head> следующую строку:

```
<head>
...
<link rel="stylesheet" type="text/css" href=" mysite.css">
```

...
</head>

В этой строке указывается, что связываемый файл является таблицей стилей (rel=stylesheet), формат этого файла – .css1 (type=text/css) и находится он в той же директории, что и файл html, либо имеет другой URL-адрес (href= mysite.css). Очевидно, что эту строку мы можем прописать в любом (либо во всех) из наших html-файлов. Таким образом, единое стилевое оформление будет прописано для нескольких страниц сразу.

Селекторы CSS

Селектор class

Допустим, мы хотим создать страницу, на которой будет два вида абзацев <p>, причем оба вида будут постоянно чередоваться и часто повторяться. Типичный пример такой страницы – интервью, в котором чередуются вопросы журналиста и ответы интервьюируемого. Естественно, при создании такой страницы мы захотим визуально отделить вопросы и ответы друг от друга. Мы можем создать в одной таблице стилей два различных класса абзацев с помощью селектора класса. Это будет выглядеть так:

```
<html>
<head>
...
<style>
...
p.ask {
font-style: italic;
font-weight: bold;
font-family: Arial, sans-serif;
font-size: 10pt;
color: gray;
margin-left: 15px
}
p.answer {
font-family: 'Times New Roman', serif;
font-size: 12 pt; color: black;
}
...
</style>
...
</head>
<body>
...
<p class="ask">Вопрос журналиста</p>
<p class="answer">Ответ интервьюируемого</p>
...
```

```
</body>
</html>
```

В приведенном примере вопросы журналиста будут отображаться шрифтом Arial серого цвета, полужирным, курсивного начертания, размером 10 пунктов с отступом 15 пикселов от левого края страницы. Ответы же будут отображены шрифтом Times New Roman размером 12 пунктов черного цвета. Важно не забывать прописывать параметр class различным классам абзацев непосредственно в коде html. Вы можете создавать любое количество классов для любых элементов страницы.

Селектор id

Возьмем другой случай. Предположим, вы хотите создать на странице какие-либо уникальные элементы, к которым в дальнейшем планируете обращаться из программ JavaScript. Возможно, эти элементы будут повторяться на других страницах, и вы хотели бы задать им единое оформление посредством CSS. На этот случай в каскадных таблицах стилей имеется возможность присвоения уникальным элементам идентификаторов (id). Наиболее часто идентификаторы используются для элементов форм, которые в спецификации HTML 4.0 имеют полную либо ограниченную поддержку CSS (в зависимости от элемента). Вот пример назначения идентификатора и правил CSS таким элементам:

```
<html>
<head>
...
<style>
...
input#green {color: green;}
input#red {color: red;}
...
</style>
...
</head>
<body>
...
<form ...>
<p>Текст, вводимый в это поле, будет отображен зеленым цветом:
<input type="text" id="green" name="info1" size="20"></p>
<p>Текст, вводимый в это поле, будет отображен красным цветом:
<input type="text" id="red" name="info2" size="20"></p>
</form>
...
</body>
</html>
```

Аналогичным образом уникальные идентификаторы могут быть назначены любому количеству любых элементов страницы. Это может быть полезно для

обращения к элементу из программы на языке JavaScript и изменения стиля его отображения в ответ на действия пользователя, что позволяет создавать различные динамические эффекты.

Контекстные селекторы

Допустим, мы создали таблицу стилей, согласно которой все заголовки первого уровня на странице отображаются красным цветом на сером фоне, все абзацы – зеленым цветом на желтом фоне, а все выделенные посредством тега слова внутри абзацев – черным цветом на серебристом фоне. Код таблицы стилей, как вы уже догадались, при этом выглядит так:

```
h1 {
color: red;
background-color: gray;
}
p {
color: green;
background-color: yellow;
}
em {
color: black;
background-color: silver;
}
```

Предположим, мы хотим также выделить какие-то слова в заголовке посредством того же тега , но нас не устраивает появление черного цвета текста на серебристом фоне в заголовке. Мы хотим выделить слова в заголовке бордовым цветом на сером фоне. Для этого существуют контекстные селекторы. Запись правила, которое мы для этого добавим в таблицу стилей, будет выглядеть так:

```
h1 em {
color: #CC0000;
background-color: gray;
}
```

А вот пример кода страницы с использованием этого контекстного селектора:

```
<html>
<head>
...
<style>
...
h1 {
color: red;
background-color: gray;
}
p {
color: green;
background-color: yellow;
}
```

```
em {
color: black;
background-color: silver;
}
h1 em {
color: #CC0000;
background-color: gray;
}
...
</style>
...
</head>
<body>
...
<h1>Это – заголовок первого уровня с <em>выделенным</em>
словом</h1>
<p>А это – обычный абзац с <em>выделенными словами</em>
</p>
...
</body>
</html>
```

Последний тип стилей – это возможность стилизовать элементы в зависимости от их состояния. Прямым примером этого является стиль ссылок. Когда мы создаём ссылку, мы должны нацелить элемент <a> (якорь). Он имеет различные состояния в зависимости от того, посещается ли он, посещается, находится над ним, фокусируется с помощью клавиатуры или в процессе нажатия (активации). Вы можете использовать CSS для нацеливания на эти разные состояния – CSS-код ниже отображает невидимые ссылки розового цвета и посещенные ссылки зелёного цвета.

```
a:link {
  color: pink;
}

a:visited {
  color: green;
}
```

Вы можете изменить внешний вид ссылки, когда пользователь наводит на неё курсор, например, удалив подчеркивание, что достигается с помощью следующего правила:

```
a:hover {
  text-decoration: none;
}
```

Стоит отметить, что Вы можете комбинировать несколько селекторов и комбинаторов вместе. Вот пример:

```
/* выбирает любой <span> внутри <p>, который находится внутри
<article>  */
article p span { ... }

/* выбирает любой <p>, который идет сразу после <ul>, который идет
сразу после <h1>  */
h1 + ul + p { ... }
```

Вы также можете комбинировать несколько типов вместе. Попробуйте добавить следующее в ваш код:

```
body h1 + p .special {
  color: yellow;
  background-color: black;
  padding: 5px;
}
```

Это будет стиль любого элемента с классом special, который находится внутри <p>, который приходит сразу после <h1>, который находится внутри <body>.

В оригинальном HTML, который мы предоставили, единственный элемент в стиле .

Как и в любом достаточно сложном языке, при создании таблицы стилей можно пользоваться комментариями:

/ Этот текст является комментарием */*

Все параметры, используемые для определении стиля, условно можно разделить на несколько больших групп: управляющие видом шрифта (гарнитура, кегль, цвет, наклон, жирность,..) управляющие форматированием абзаца (выравнивание, интерлиньяж, расстояние между словами, отступ красной строки,..) управляющие свойствами блока (отступы слева-сверху-справа-снизу, местоположение блока на страничке, видимость блока,..) другие, которые не вписываются ни в одну из перечисленных выше групп (цвет ссылок странички, изменение внешнего вида курсора). Рассмотрим подробнее параметры, используемые для управления внешним видом текста и форматирования абзацев - как наиболее часто употребляемые.

Основные параметры шрифта

- font-weight: [*bold*|*normal*|*число*] – жирность шрифта
- font-style: [*normal*|*italic*|*oblique*] – наклон шрифта
- font-size: *число* – размер шрифта
- font-family: *имя шрифта* – гарнитура шрифта
- color: *число* – цвет шрифта
- background-color: *число* – цвет подложки
- background: *url* – текстурная подложка

Псевдоклассы ссылок

– A:active{} Таблица стилей для активных ссылок (при нажатии)

– A:link{} Таблица стилей для собственно ссылок

– A:visited{} Таблица стилей для посещённых ссылок

– A:hover {} Таблица стилей для ссылок при наведении указателя мыши

Основные параметры абзаца (а также элементов типа "Box" (div-блоки))

– text-align: [*left*|*right*|*center*|*justify*] – выравнивание

– text-indent: *число* – отступ красной строки

– line-height: *число* – интерлиньяж

– letter-spacing: *число* – трекинг

– padding-left: *число* – отступ от текста слева

– padding-right: *число* – отступ от текста справа

– padding-top: *число* – отступ от текста сверху

– padding-bottom: *число* – отступ от текста снизу

– margin-left: *число* – отступ от границы слева

– margin-right: *число* – отступ от границы справа

– margin-top: *число* – отступ от границы сверху

– margin-bottom: *число* – отступ от границы снизу

В свойствах, которым требуется указание размеров, можно использовать несколько способов для их задания:

– относительный размер в процентах (%);

– относительный размер при помощи словесного описания (larger, smaller, xx-small, x-small, small, medium, large, x-large, xx-large);

– абсолютный размер в типографских единицах – размер может задаваться в пунктах (pt), пиках (pc), пикселях (px), средней шириной буквы "m" (em), средней шириной буквы "x" (ex);

– абсолютный размер в стандартных единицах длины – размер может задаваться в сантиметрах (cm), миллиметрах (mm), дюймах (in) абсолютный в пикселях (px).

Цвет для тех свойств, где это нужно, может быть определен одним из трех способов:

– при помощи названия цвета: yellow, red, green, grey, ...

– шестнадцатеричным заданием цвета в формате #RRGGBB: #ff0000, #883490, #ffffff, …

– десятичным заданием составляющих цвета в формате rgb(red, green, blue): rgb(255,0,0), rgb(100,23,78).

Общая постановка задачи

1. В соответствие с выбранной из списка индивидуальных данных (см. ниже) темой создать сайт, состоящий как минимум из 4-х страниц, оформленных с использованием css. Каждая страница сайта должна

содержать меню, позволяющее переходить на другие страницы. Для главной страницы должен быть создан отдельный файл со стилем. Все остальные страницы оформляются с помощью других стилей css (возможно применение одного стиля для всех страницы кроме главной). При оформлении **обязательно** используйте псевдоклассы, селекторы id и class.

2. Разместить разработанный сайт на FTP-сервере в личной папке в подкаталоге с названием, соответствующим выбранной теме. Открытие сайта на сервере должно происходить путем перехода по URL в браузере.

3. Отправить в качестве отчета преподавателю URL сайта для проверки.

Список индивидуальных данных

1. Создать сайт кадрового агентства с возможностью подачи анкеты соискателя на вакантную должность.
2. Создать сайт туристической фирмы с возможностью поиска интересующего тура.
3. Создать сайт интерактивного абонемента библиотеки с возможностью бронирования книги.
4. Создать сайт пиццерии с возможностью оформления заказа пиццы.
5. Создать сайт оператора мобильной связи с возможностью оплаты услуги связи.
6. Создать сайт булочной с возможностью добавления нового товара.
7. Создать сайт РЭУ с возможностью оформления заявки на ремонт.
8. Создать сайт кинотеатра с возможностью бронирования билета.
9. Создать сайт брачного агентства с возможностью подачи анкеты соискателя.
10. Создать сайт банка с возможностью открытия клиентского счета.
11. Создать сайт салона красоты с возможностью бронирования получения услуг.
12. Создать сайт автовокзала с возможностью бронирования билета на автобус.
13. Создать сайт учебной группы с возможностью добавления анкеты студента.
14. Создать сайт аэропорта с возможностью бронирования билета на самолет.
15. Создать сайт общежития с возможностью добавления анкеты проживающего.
16. Создать сайт факультета с возможностью добавления истории студента.
17. Создать сайт молокозавода с возможностью оформления заказа на молочную продукцию.
18. Создать сайт магазина игрушек с возможностью оформления заказа на игрушку.

19. Создать сайт политической партии с возможностью проведения опроса населения.

20. Создать сайт частной школы с возможностью подачи заявки на зачисление.

21. Создать сайт больницы с возможностью добавления данных осмотра больного.